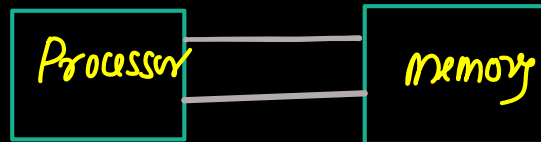


Memory systems. (Note)

- A computer based system consists of a processor and memory.
- memory store the information, which include data and instruction



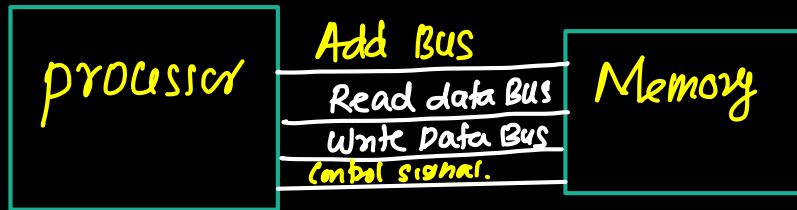
- processor fetch the instruction from the memory, decode then and execute them. During the execution, the CPU also reads from memory and write the result back to the memory.
- processor communicates with memory using a memory interface consists of buses and control signals.
- Two fundamental operations performed on memory are read and write

Read operation

- processor places address on address bus
- processor asserts read control signal
- memory system return the data on the read data bus.

Write operation

- processor place the address on the address Bus
- processor place the data on the write data Bus
- processor assert control signal
- memory store the data



Speed Gap btw processor and Memory.

- processor is a very fast device
- processor speed is given by clock frequency.

e.g. If $f_c = 2\text{GHz} = 2 \times 10^9 \text{ Hz}$

$$T = \frac{1}{f_c} = \frac{1}{2 \times 10^9} = \underline{0.5 \text{ ns}}$$

i.e. clock cycle time = 0.5 ns.

CPU can complete one task in 0.5 ns.

- Processor needs to access the memory, But memory access time is much larger value. i.e. memory is comparatively slow device.
- CPU waste many cycles for memory.
- We need to bridge the gap processor and memory.

Design factor of memory

While designing the memory system, three important factors are considered

- ① access time
- ② Cost
- ③ Memory size

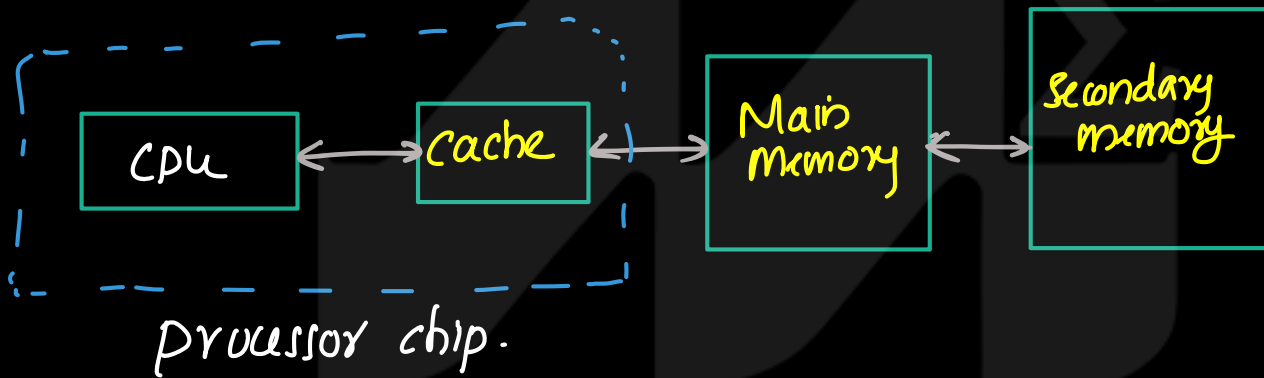
In general Faster memory is more expensive and smaller size
Larger memory is cheaper but slower

Memory hierarchy

- Memory hierarchy is used to bridge gap between the fast CPU and slow large memories. by organizing storage into different levels.
- Most frequently used information is kept in the fastest memory closest to the CPU

memories

- ① Cache
- ② Main Memory
- ③ Secondary memory.



- the data and instructions that are required during the processing of data are brought from secondary memory and stored in the main memory.
- During the process, instructions data from the main memory should bring to CPU. It will take more time and affect the speed of processing if computer
- So a high speed memory placed b/w CPU and main memory. Cache memory store more frequently used data and make it available to CPU at a fast rate.
- During the process, CPU first check cache for the required data. If data is not found in cache then it look at the Main memory. and execute the instructions

→ If required information is not present in the main memory, the operating system brings the required information from secondary to main memory. CPU takes the data and executes the instruction. (N)

- Computer stores the most commonly used instructions and data in a faster but smaller memory called cache. Cache is usually built on the same chip as the processor. Cache speed is comparable to the processor speed.
- If the processor requests data that is available in the cache, it is returned quickly. This is called a cache hit. Otherwise, the processor retrieves the data from main memory. This is called a cache miss.

M SIGMA
GOKULAM

Temporal Locality and Spatial Locality.

Note

Temporal locality.

- Temporal locality means that if a memory location (data or instruction) is accessed once, then it is likely to be accessed again within a short period of time.
- So keep the content of recently accessed memory location (data/instruction) in higher level of memory (cache)

Spatial Locality

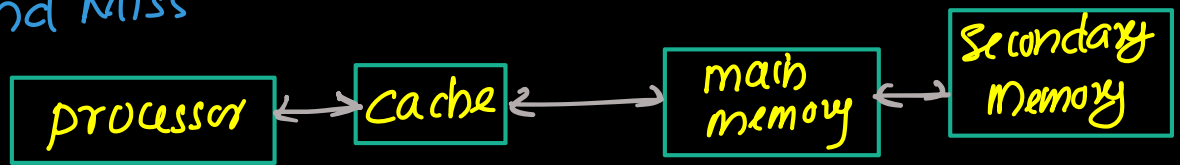
- Spatial locality means that, if a memory location is accessed, then near by memory locations are likely to be accessed soon.
- When a data accessed from the memory location, bring data from near by memory location into higher level of memory hierarchy.

M SIGMA
GOKULAM

Memory Performance (Note)

- The performance of a memory system is measured using the following parameter:
① Hit Rate ② Miss Rate ③ average memory access time.

Hit and Miss



- When processor requests instructions/data, it first checks the memory closest to CPU (Cache). If requested information is present in the memory (Cache), then it is called cache hit. In this case CPU gets the data quickly.
- If requested information is not found in the memory (Cache), then it is called miss (Cache Miss). Then data must be fetched from low level memory (Main memory).

If Requested information present in the memory, then it is called Hit. Then processor gets the information quickly.

If Requested information is not present in the memory, then it is called Miss. Then fetch the information from the low level memory.

Hit Rate (H)

$$\text{Hit Rate} = \frac{\text{number of hits}}{\text{Total number of memory access}}$$

$$\text{Hit Rate} = 1 - \text{Miss Rate}$$

Miss Rate

(2)

$$\text{Miss Rate} = \frac{\text{number of misses}}{\text{Total number of memory access}}$$

$$\text{Miss Rate} = 1 - \text{Hit Rate}$$

Suppose a program has 2,000 data access instructions (loads or stores) and 1,250 of these requested data values are found in the cache. The other 750 data values are supplied to the processor by main memory. What are the miss and hit rates for the cache?

$$\text{Total No. of memory access} = 2000$$

$$\text{No. of hits (cache)} = 1250$$

$$\text{no. of Misses (cache)} = 750$$

$$\text{Hit Rate} = \frac{\text{No. of Hit}}{\text{Total no. of mem access}} = \frac{1250}{2000} = 0.625 = \underline{\underline{62.5\%}}$$

$$\text{Miss Rate} = 1 - \text{HR} = 1 - \frac{1250}{2000} = \underline{\underline{37.5\%}}$$

(2) Average Memory access time

Consider a 3 level memory system (M_1, M_2, M_3) with hit ratio H_1, H_2, H_3 respectively. access time for each memory is t_1, t_2, t_3

cache

main
Memory

secondary
memory.

Hit Ratio H_1

H_2

$H_3 = 1$

Time t_1

t_2

t_3 .

→ CPU request data, if it is not present (miss) in the cache, then check in the main memory. If data is not present in the main memory, then check in the secondary Memory. data will be available in the

Secondary memory so hit rate ($H_3 = 1$).

(N)

→ Average memory access time (AMAT)

$$\begin{matrix} H_1 & H_2 & H_3 = 1 \\ t_1 & t_2 & t_3 \end{matrix}$$

$$AMAT = \left(\frac{H}{H} \right) \left(\frac{t}{t} \right) + \left(\frac{H}{H} \right) \left(\frac{t}{t} \right) + \left(\frac{H}{H} \right) \left(\frac{t}{t} \right)$$

$$= H_1 t_1 + (1-H_1) H_2 [t_1 + t_2] + (1-H_1) (1-H_2) H_3 (t_1 + t_2 + t_3)$$

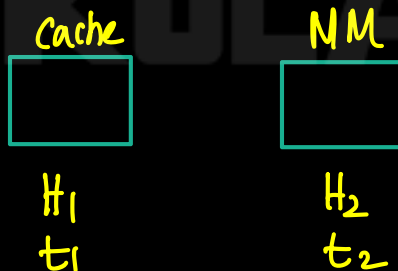
$$AMAT = H_1 t_1 + (1-H_1) H_2 (t_1 + t_2) + (1-H_1) (1-H_2) (t_1 + t_2 + t_3)$$

Average memory access time is the average time the CPU spends to get data/instruction from the memory system.

Suppose a computer system has a memory organization with only two levels of hierarchy, a cache and main memory. What is the average memory access time given the access times and miss rates in Table

Table 8.1 Access times and miss rates

Memory Level	Access Time (Cycles)	Miss Rate
Cache	1	10%
Main Memory	100	0%



$$H_1 = 1 - \text{miss rate} = 1 - 0.1 = \underline{0.9}$$

$$H_2 = 1 - \text{miss rate} = 1 - 0 = \underline{1}$$

$$t_1 = 1 \text{ (cycle)}$$

$$t_2 = 100 \text{ (cycle)}$$

$$AMAT = H_1 t_1 + (1-H_1) (H_2) (t_1 + t_2)$$

$$= 0.9 \times 1 + 0.1 \times 1 \times 101 = 0.9 + 10.1 = 11 \text{ cycles.}$$

A system has a Cache access time of 2ns and a main memory access time of 100ns. If the Hit Ratio is 95%, calculate the AMAT

(N)

processor

cache

MM

$$T_1 = 2ns$$

$$T_2 = 100ns$$

$$H_1 = 0.95$$

$$H_2 = 1$$

$$\begin{aligned} AMAT &= H_1 T_1 + (1 - H_1) H_2 (T_1 + T_2) \\ &= 0.95 \times 2 \times 10^{-9} + 0.05 \times 1 \times (2 + 100) \times 10^{-9} \\ &= (1.9 + 5.1) \times 10^{-9} \\ &= \underline{\underline{7 \times 10^{-9}}} \end{aligned}$$



Cache Memory (Note)

- Cache memory is fast memory placed close to the processor
- it holds the commonly used data from the main memory so that processor can access it quickly.
- Cache can store both instruction and data. (most frequently used and recently used).

— how data found in cache



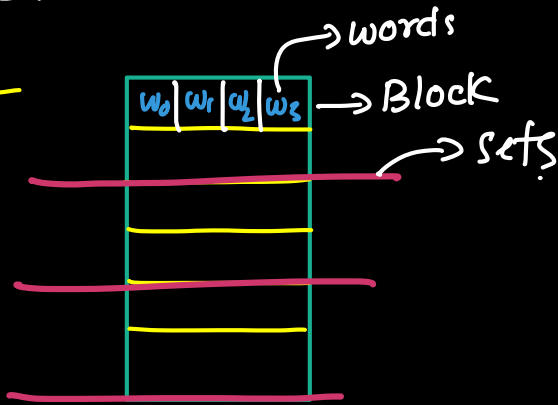
When a processor needs an instruction/data

- The CPU first checks the cache
- If the required data is present in the cache (cache hit), then processor accesses data.
- If the required data is not present in the cache, then CPU fetches from the main memory and copies to the cache for future use.
- If processor accesses the data (information) recently, it is likely to access it again soon. So fetched data is copied to the cache. So the future access will likely hit.
- If processor accesses the data from a memory location, it is likely to access a nearby location soon. When cache fetches one word from main memory, it may also fetch several adjacent words.
- The group of words are called blocks.
- no. of words in the block is called block size

Cache organisation

— A Cache is organised into several sets. Each set can hold one or more Blocks of data

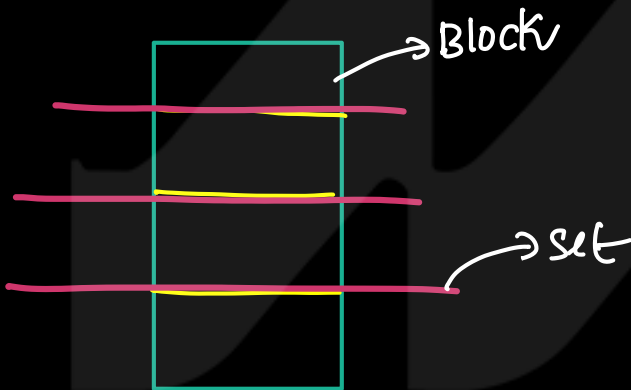
eg: 1



→ Here one sets consists of 2 Block.

no: of Block in a set = 2

eg: 2 One Block in set



eg: Cache size = 16 Byte = 16 word

1 Byte = 1 word

no: of word in Block (Block size) = 4 Byte = 4 word

$$\text{No: of Blocks} = \frac{\text{Cache size}}{\text{Block Size}} = \frac{16 \text{ word}}{4 \text{ word}} = \underline{\underline{4}}$$

no: of blocks in set = 1

No: of set = 4

no: of bits to represent set = $\log_2 \text{no: of set}$

$$= \log_2 4 = \underline{\underline{2 \text{ bit}}}$$

no: of bits to represent words inside the Block = $\log_2 (\text{no: of words in a Block})$

$$= \log_2 4 = \underline{\underline{2 \text{ bit}}}$$

Cache terminology.

(N)

① Capacity (C)

— no: of data bytes in the cache

→ eg: $C = 64 \text{ Byte}$.

eg: $1 \text{ Byte} \rightarrow 1 \text{ word}$

$C = 64 \text{ word}$

— Block size (b)

Block $\rightarrow$ group of words

Block size — no: of words in a Block

eg: $b = 4 \text{ word}$

it contains 4 words

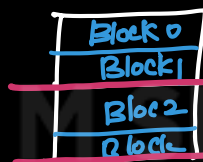
→ no: of Block (B)

$$B = \frac{\text{Capacity (C)}}{\text{no: of block (b)}} = \frac{C}{b}$$

→ Degree of associative (N)

no: of Blocks in a set

eg:



— Set No 0

— Set No 1

$N = 2$ (1 set contains 2 Block)

→ No: of sets (S)

$$S = \frac{\text{no: of Block}}{\text{no: of Block in a set}} = \frac{B}{N}$$

$$S = B/N$$

eg: Main Memory size = 4GB

Cache size = 1MB

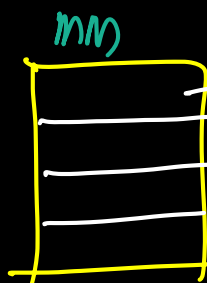
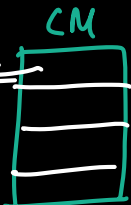
Block size = 4KB.

find physical address

2 way set - cache

Steps. $\rightarrow$

Cache Mem
Block



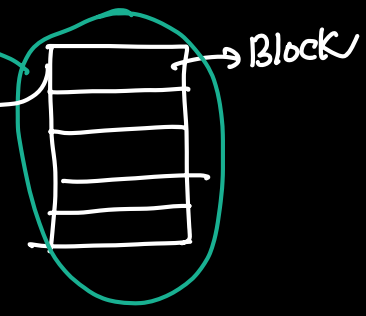
main Memory Block

(N)

① Find the No: of Block in cache (B)

$$B = \frac{\text{Cache size}}{\text{block size}}$$

$$B = \frac{C}{b}$$



$$C = 1MB = 1 \times 2^{20} B$$

$$1B \rightarrow 1 \text{ word}$$

$$C = 1 \times 2^{20} \text{ word}$$

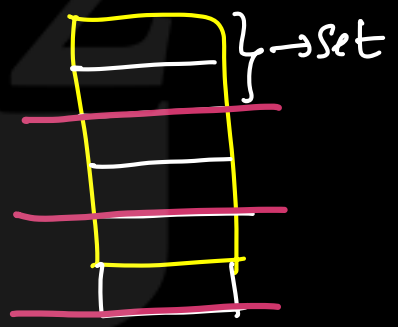
$$b = 4KB = 4 \times 2^{10} \times \text{word} = 2^{12} \text{ word}$$

$$B = \frac{2^{20}}{2^{12}} = \underline{\underline{2^8}}$$

$$\text{no: of bits to Rep } B = \log_2 2^8 = \underline{\underline{8}} \text{ bits}$$

2-way Set $\rightarrow$ 2 Block in 1 set

$$\text{no: of set } (S) = \frac{\text{no: of Block } (B)}{\text{no: of Block in a set } (N)}$$

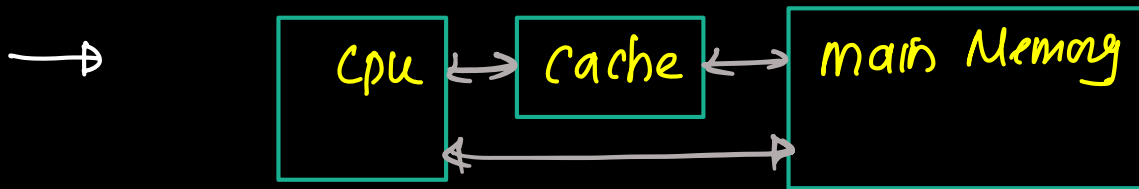


$$S = \frac{2^8}{2} = \underline{\underline{2^4}}$$

$$\text{no: of bits to represent set} = \log_2 2^4 = \underline{\underline{4}}$$

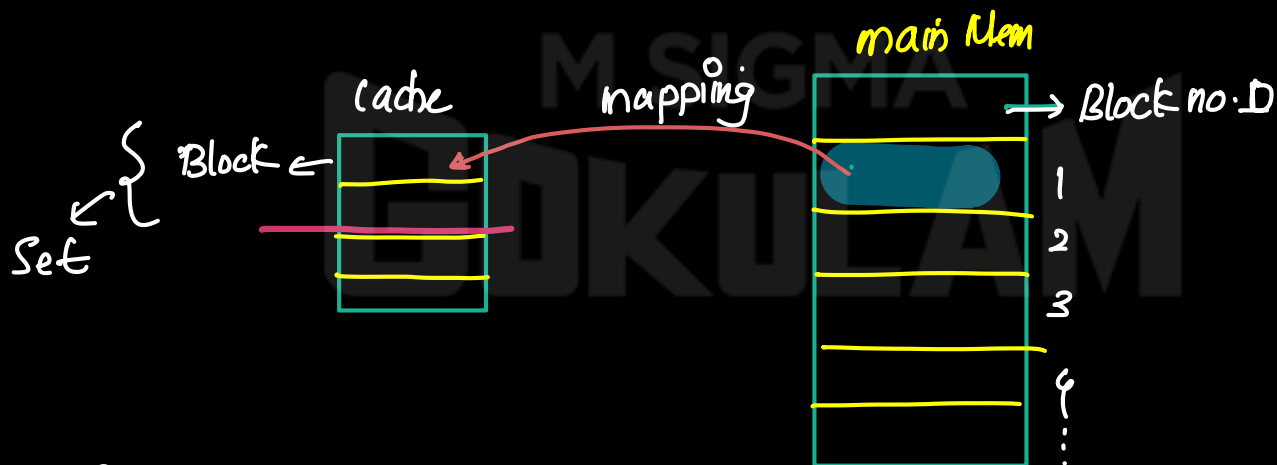
GOKULAM

Cache Mapping technique. (Note)

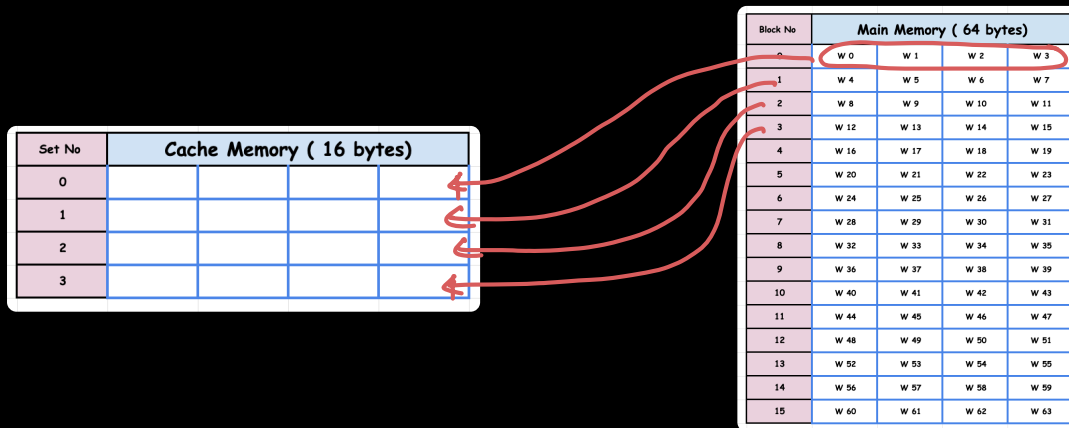


— When CPU access information (data/Instruction) from the main memory, a copy of that information is placed in the cache memory. assuming that same data will be used again in the near future. Not only the requested information, but also the information stored in the neighbouring memory location is brought into the cache. This is because of the principle of locality of Reference.

— Main memory is divided into Blocks and cache memory divided into sets. each block of main memory must be mapped to the same location of cache memory.



→ Size of cache memory is limited when compared to main memory. So all memory blocks cannot be stored in the cache memory at same time. , a technique is required to decide how blocks of main memory are place into cache memory and how CPU locates the required data in the cache. This technique is called Cache mapping technique.



Consider a cache with 4 set (one block in one set).

Block no: 0 of MM is mapped to set no 0

$B_0 \rightarrow S_0$
 $B_1 \rightarrow S_1$
 $B_2 \rightarrow S_2$
 $B_3 \rightarrow S_3$

Where we place remaining blocks . so we need Rule to show how we can place main memory Blocks in cache

Definition of Cache Mapping

Cache mapping is the technique used to determine how blocks of main memory are placed into cache memory and how the CPU locates the required data in the cache.

Need for Cache Mapping

- Main memory is large but slow
Accessing data directly from main memory increases CPU waiting time.

- Cache memory is small but very fast
Frequently used data can be accessed quickly if stored in cache.

- Limited cache size
Since cache cannot hold all memory blocks, a proper mapping method is required.

- Improved system performance
Cache mapping reduces average memory access time and improves overall CPU performance.

Types of Cache Mapping

Note

- ① Direct mapping (one block in a set)
- ② Fully associative mapping (only one set & all blocks are in a single set)
- ③ Set associative mapping (2 or more blocks in a set)

Direct mapping

Cache memory is organised into a large number of sets. In direct mapping cache, each set contains only one block. It is a cache mapping technique in which each block of main memory can be stored in exactly one specific set in the cache.

Example Explanation

Consider the following:

- Main memory size = 64 bytes
- Block size = 4 bytes

$$\text{Number of blocks in main memory} = \frac{64}{4} = 16 \text{ blocks}$$

- Cache size = 16 bytes
- Cache block size = 4 bytes (same as main memory block size)

$$\text{Total blocks in cache} = \frac{16}{4} = 4 \text{ blocks}$$

Cache Organisation

- Since this is direct mapping, one set consists of one block
- Therefore:

$$\text{Number of sets in cache} = \text{Number of cache blocks} = 4$$

- Cache sets are:

- Set 0
- Set 1
- Set 2
- Set 3

Set No	Cache Memory (16 bytes)			
0				
1				
2				
3				

Block No	Main Memory (64 bytes)			
0	W 0	W 1	W 2	W 3
1	W 4	W 5	W 6	W 7
2	W 8	W 9	W 10	W 11
3	W 12	W 13	W 14	W 15
4	W 16	W 17	W 18	W 19
5	W 20	W 21	W 22	W 23
6	W 24	W 25	W 26	W 27
7	W 28	W 29	W 30	W 31
8	W 32	W 33	W 34	W 35
9	W 36	W 37	W 38	W 39
10	W 40	W 41	W 42	W 43
11	W 44	W 45	W 46	W 47
12	W 48	W 49	W 50	W 51
13	W 52	W 53	W 54	W 55
14	W 56	W 57	W 58	W 59
15	W 60	W 61	W 62	W 63

→ Block no 0 (b_0) in the main memory mapped to set no '0' (s_0) in the cache memory.

$s_0 \xleftarrow{\text{mapped}} b_0$
 $s_1 \xleftarrow{\quad} b_1$
 $s_2 \xleftarrow{\quad} b_2$
 $s_3 \xleftarrow{\quad} b_3$

Set No	Cache Memory (16 bytes)			
0				
1				
2				
3				

Block No	Main Memory (64 bytes)			
0	W 0	W 1	W 2	W 3
1	W 4	W 5	W 6	W 7
2	W 8	W 9	W 10	W 11
3	W 12	W 13	W 14	W 15
4	W 16	W 17	W 18	W 19
5	W 20	W 21	W 22	W 23
6	W 24	W 25	W 26	W 27
7	W 28	W 29	W 30	W 31
8	W 32	W 33	W 34	W 35
9	W 36	W 37	W 38	W 39
10	W 40	W 41	W 42	W 43
11	W 44	W 45	W 46	W 47
12	W 48	W 49	W 50	W 51
13	W 52	W 53	W 54	W 55
14	W 56	W 57	W 58	W 59
15	W 60	W 61	W 62	W 63

→ Now block no 4 is mapped to set no '0'

$s_0 \xleftarrow{\text{mapped}} b_4$
 $s_1 \xleftarrow{\quad} b_5$
 $s_2 \xleftarrow{\quad} b_6$
 $\vdots$

→ Mapping function

$$\text{Set no} = (\text{Block no}) \bmod (\text{no. of sets in the cache})$$

For Block no 0 $\rightarrow 0 \% 4 = 0 \rightarrow \text{Set no} = 0$

$b_0 \rightarrow 0 \% 4 = 0 \rightarrow s_0$

$b_1 \rightarrow 1 \% 4 = 1 \rightarrow s_1$

$b_2 \rightarrow 2 \% 4 = 2 \rightarrow s_2$

$b_3 \rightarrow 3 \% 4 = 3 \rightarrow s_3$

$b_4 \rightarrow 4 \% 4 = 0 \rightarrow s_0$

$b_5 \rightarrow 5 \% 4 = 1 \rightarrow s_1$

$b_6 \rightarrow 6 \% 4 = 2 \rightarrow s_2$

$b_7 \rightarrow 7 \% 4 = 3 \rightarrow s_3$

$b_8 \rightarrow 8 \% 4 = 0 \rightarrow s_0$

$\vdots$

$b_{11} \rightarrow 11 \% 4 = 3 \rightarrow s_3$

$$b_{15} \longrightarrow 15 \div 4 = 3$$

20

Set no	Block.
S_0	b_0, b_4, b_8, b_{12}
S_1	b_1, b_5, b_9, b_{13}
S_2	b_2, b_6, b_{10}, b_{14}
S_3	b_3, b_7, b_{11}, b_{15}



Address format in direct mapping

A physical Address is divided into 3 field

(N)

tag bits	Set bits	Block offset
----------	----------	--------------

- ① Block offset field
- ② Set bit field
- ③ tag bits.

Set bit field

→ these bits are used to identify which set no should be accessed

→ tag bit field

these bits are used to identify the which Block in the main memory is stored in the selected set

→ Block offset

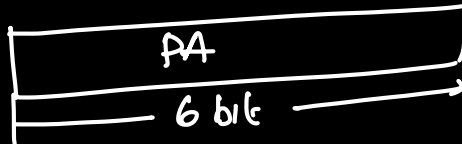
these bits are used to identify the which word inside the blocks should be needed.

MM size = 64 Byte = 64 word

1 Byte = 1 word
Block size = 4 word, Cache size = 16 word

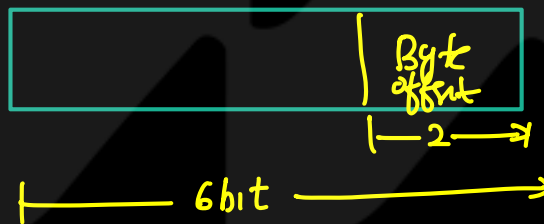
Step 1 → Find physical Address bits.

MM size = 64 Byte = 64 word
No. of bits to represent MM = $\log_2 64 = 6$ bits



Step 2 → Find the Block offset

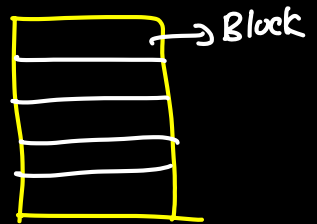
Block size = 4 word
No. of bits to represent words inside the block = Block offset
Block offset = $\log_2 4 = 2$ bits



Step 3 — Find Set bit

Find the no. of sets in the cache.

$$\begin{aligned} \text{No. of Block in cache} &= \frac{\text{Cache size}}{\text{Block size}} \\ &= \frac{16 \text{ word}}{4 \text{ word}} = \underline{\underline{4}} \end{aligned}$$



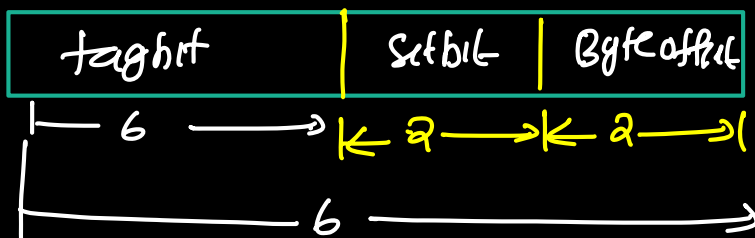
No. of block in a set = 1

Total No. of set = No. of Block = 4

Set bit = $\log_2 4 = \underline{\underline{2}}$ bit

Find tag bit

tag bit = PA bit - Set bit - Byte offset



How the CPU Accesses Data from Cache (Note)

1. Processor generates a physical address (PA)

When the CPU needs data or an instruction, it generates a physical address.

This physical address is divided into three fields:

- Tag
- Set index (set bits)
- Block/offset

2. Cache organization into sets

- The cache is divided into multiple sets.
- Each set contains one blocks)
- Every Set in the cache has:
 - Tag field - identifies which block of main memory is stored
 - Valid bit - indicates whether the data in that line is valid
 - Data field - actual data copied from main memory

3. Selecting the set using set bits

- The CPU uses the set bits from the physical address to select one specific set in the cache.
- These set bits determine which set may contain the required block of main memory.

4. Tag comparison

- Within the selected set, the tag field from the physical address is compared with the tag stored in the set.
- This comparison checks whether the required memory block is present in that set.

5. Valid bit check

- At the same time, the valid bit of the selected set is checked.
- If the valid bit is 1, it means the data stored in that set is valid.

6. Cache hit condition

A cache hit occurs if:

- The tag bits match, and
- The valid bit is 1

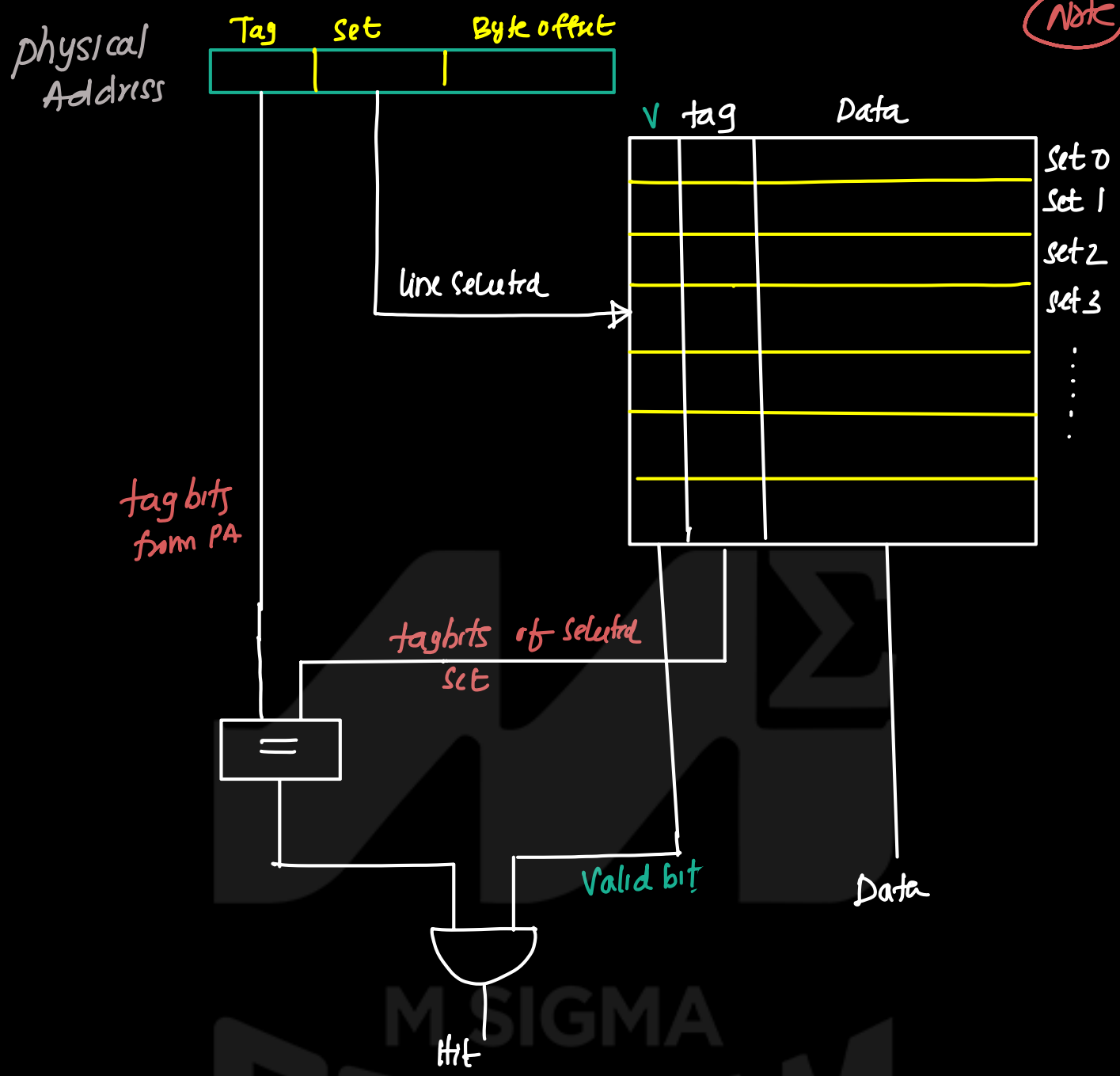
7. Data access

- When a cache hit occurs, the CPU directly copies the required data from the data field of the selected set.
- This is much faster than accessing main memory.

8. Cache miss (for completeness)

- If the tag does not match or the valid bit is 0, a cache miss occurs.
- The required block is then fetched from main memory, placed into the cache, the tag is updated, and the valid bit is set to 1.

Note



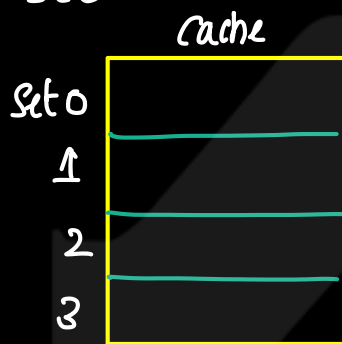
Disadvantages of Direct mapping

(N)

- In direct mapping, each memory block is mapped to exactly on one specific cache set.
- Because of this fixed mapping. multiple main memory blocks may map to same set, which leads to block conflict

Consider a main memory with 16 block and cache memory with 4 set

main memory block b_0, b_4, b_8, b_{12} are mapped to Set 0

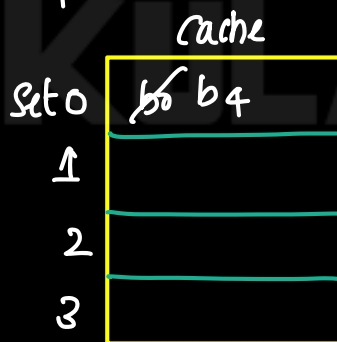


Let

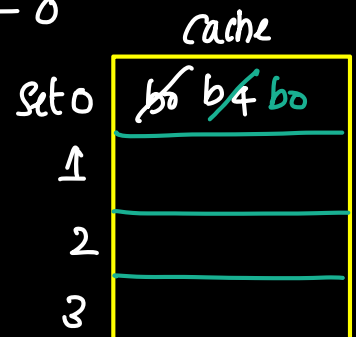
cpu block request is b_0, b_4, b_0, b_4 etc.

① first Map the block b_0 to set 0

② next block request is b_4 , so b_4 need to map to set 0. The cache replaces block b_0 and loads b_4



③ 3rd Block request is b_0 , But b_0 is not available in the cache (Miss). cache replace block 4 and maps Block b_0 to Set 0



④ 4th request is block b₄. but b₄ is not in the cache (Miss). cache replace block 0 and maps block 4 to set '0'.

⑤ Both blocks are continuously replaced in the set 0. this condition is called conflict Miss.

⑥ During the 2nd, 3rd and 4th block request blocks are not available in the cache. because all the requested blocks are mapped to same set.

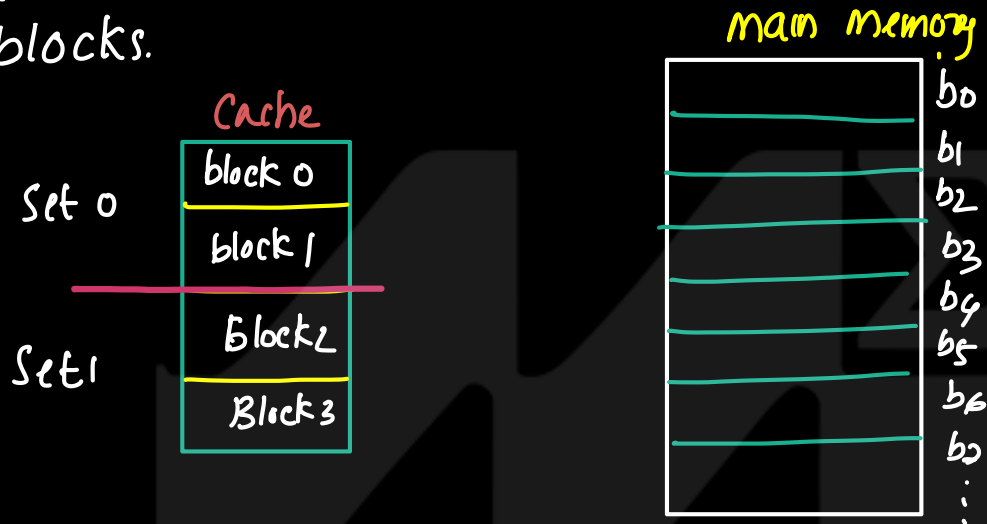
— Conflict miss can be reduced by using other mapping techniques such as set associative mapping / fully associative mapping.

M SIGMA
GOKULAM

Set associative Cache Mapping

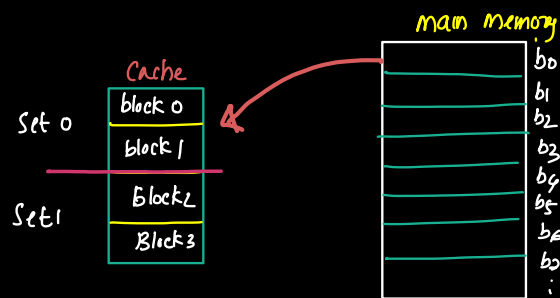
(Note)

- Explain cache mapping (Refer previous Note)
- The major problem of direct mapping is conflict miss.
- This problem can be overcome using set associative mapping
- In set associative mapping, the cache is organised into several sets. and each set contains 2 or more blocks.



→ Each block in the main memory is mapped to a particular set and memory block can be placed in any block within the particular set.

eg: Memory block '0' is mapped to set '0'. In Set 0 there are two blocks. So Memory block b0 can be mapped any block within the set 0.



- This freedom helps to overcome conflict miss.
- If N blocks are present in a cache set. Then

It is called N-associative cache

(N)

eg. If two blocks are present in a set, then it is called two way set associative mapping.

Two-way set-associative Mapping

— Consider a main memory with following specification

MM Size = 64 Byte

1 Byte = 1 word

MM Size = 64 word

MM Block size = 4 Byte = 4 word

no. of Block = $\frac{64}{4} = 16$ block.

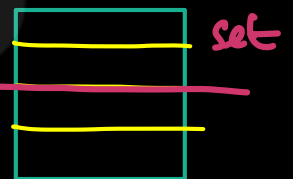
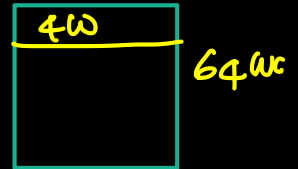
Cache size = 16 Byte = 16 word

Cache Block size = main Block size = 4B

Total cache Block = $\frac{CS}{BS} = \frac{16}{4} = 4$

No. of Block in set = 2

Total No. of Set = $\frac{\text{Total No. of Block}}{\text{No. of Block in set}} = \frac{16}{2} = 8$



Set No	block no	Cache Memory (16 bytes)
0	0	
	1	
1	2	
	3	

Block No	Main Memory (64 bytes)			
0				
1				
2				
3				
4				
5				
6				
7				
8				
9				
10				
11				
12				
13				
14				
15				

— Memory block 0 can be mapped to any block in the set 0.

mapping function

Set Number = (memory block no) mod no. of set

eg: block 15 is mapped to set no 1
 $\text{Set Num} = 15 \% 2 = 1 \rightarrow \text{set no: } 1$ (2)

→ Address field format for set associative mapping.
 A physical Address is divided into 3 field

tag bits	Set bits	Block offset
----------	----------	--------------

- ① Block offset field
- ② Set bit field
- ③ tag bits.

Set bit field

these bits are used to identify which set no should be accessed

→ tag bit field

these bits are used to identify the which block in the main memory is stored in the selected set

→ Block offset

these bits are used to identify the which word inside the blocks should be needed.

physical Address

tag bits.	Set bit	Byte offset
-----------	---------	-------------

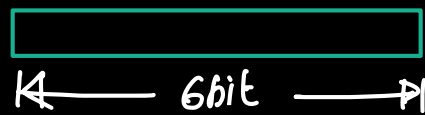
Eg: — Consider a main memory with following specification

MM Size = 64 Byte

1 Byte = 1 word

— physical Address (PA)

no. of bits in PA = $\log_2 \text{MM Size} = \log_2 64 = \underline{6} \text{ bit}$



— Block offset

$$\text{Block size} = 4 \text{ Byte} = 4 \text{ word}$$

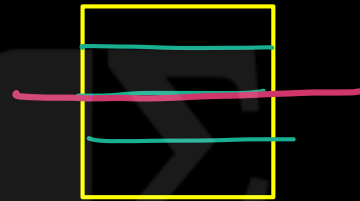
$$\text{Block offset} = \log_2 BS = \log_2 4 = \underline{2 \text{ bit}}$$



→ Set bit

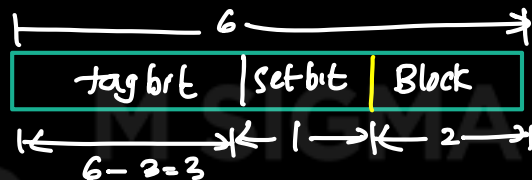
Total No: Set in cache = ?

$$\begin{aligned} \text{Total no. of block} &= \frac{CS}{\text{Block size}} \\ &= \frac{16B}{4B} = \underline{4} \end{aligned}$$



$$\text{Total No. of set} = \frac{\text{Total no. of Block}}{\text{no. of Block in set}} = \frac{4}{2} = \underline{2}$$

$$\text{Set bit} = \log_2 2 = 1 \text{ bit}$$



GOKULAM

How the CPU Accesses Data from Cache(2 way set associative)

Note

Processor generates a physical address (PA)

When the CPU needs data or an instruction, it generates a physical address.

This physical address is divided into three fields:

- Tag
- Set index (set bits)
- Block/offset

Cache organization into sets

- The cache is divided into multiple sets.
- Each set contains more than one block (let say 2)-
- Each set contains two ways
- Each way consists
 - Tag field - identifies which block of main memory is stored
 - Valid bit - indicates whether the data in that line is valid
 - Data field - actual data copied from main memory

Selecting the set using set bits

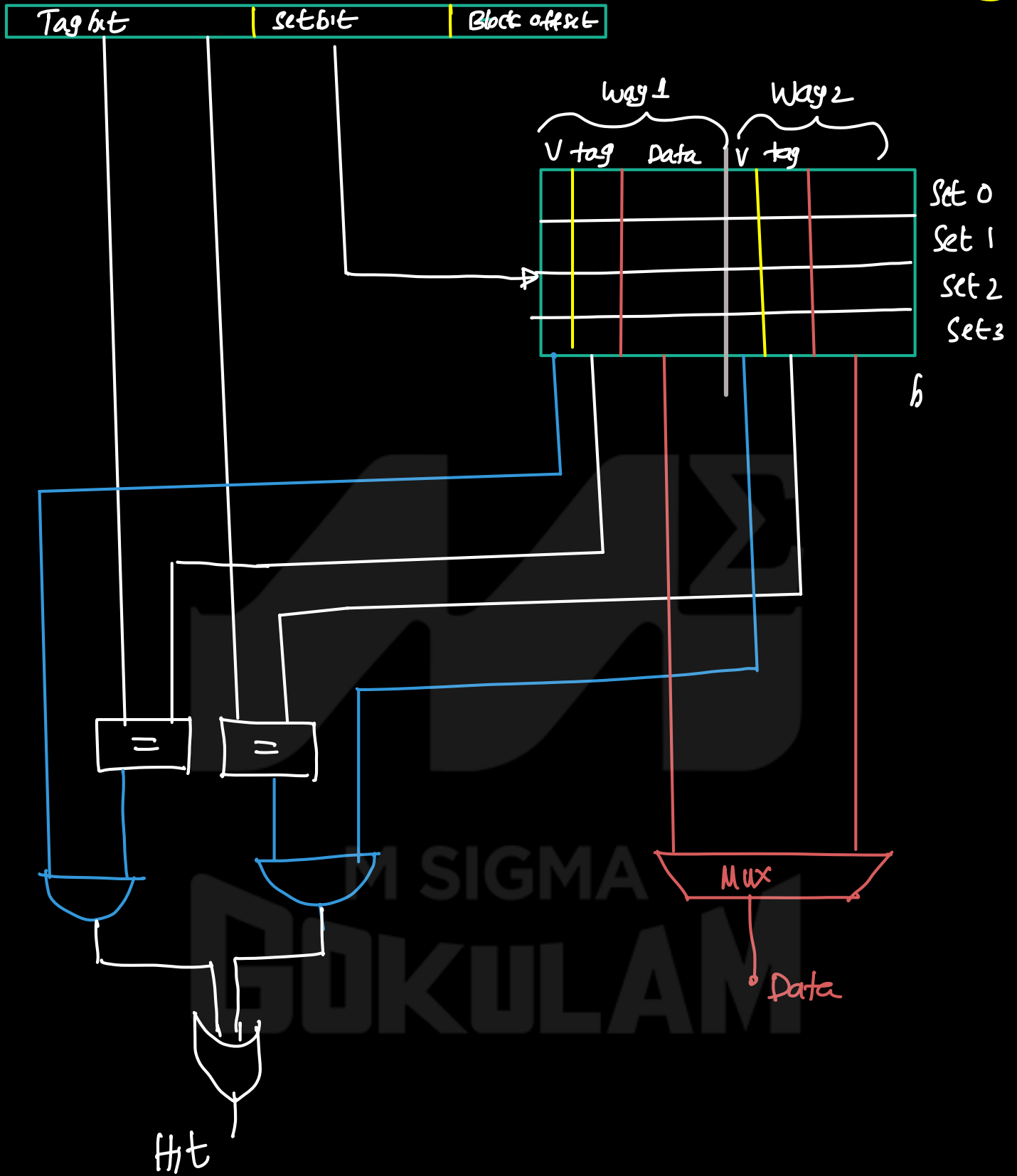
- The CPU uses the set bits from the physical address to select one specific set in the cache.
- These set bits determine which set may contain the required block of main memory.

Tag comparison

- Within the selected set, the tag field from the physical address is compared with the tag stored in the each way(2 blocks).
- This comparison checks whether the required memory block is present in any one of the way
- If the tag bits match, then check for the valid bit for the corresponding block in the selected set . if The valid bit is 1, then required data is available (cache hit)

Data access

- When a cache hit occurs, the CPU directly copies the required data from the data field of the selected set through a multiplexer.



Advantages of Set Associative Mapping

Reduces conflict misses

Since a memory block can be placed in any block within a set, conflict misses are much lower than in direct mapping.

Better cache hit ratio

Allowing multiple blocks per set increases the chance that required data is already present in the cache.

More flexible than direct mapping

Unlike direct mapping, blocks are not restricted to a single cache location.

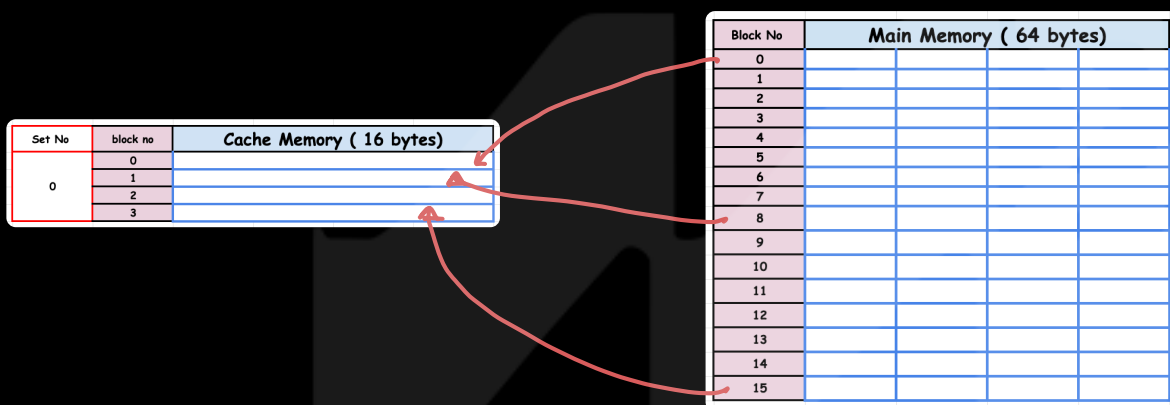


Fully Associative Mapping

- A fully associative cache contains a single set with multiple blocks

Set No	block no	Cache Memory (16 bytes)
0	0	
	1	
	2	
	3	

- Fully associative mapping is a cache mapping technique in which any block of main memory can be stored in any block of the cache. There is no fixed mapping between main memory blocks and cache blocks.



Address Format

In fully associative mapping, the physical address generated by the CPU is divided into:

Tag
Block offset

Address bits	
Tag bits	Block Offset

Cache Organisation



Each cache block contains:

- Tag field – identifies the main memory block stored
- Valid bit – indicates whether the data is valid
- Data field – stores the actual data

Working of Fully Associative Mapping

The CPU generates a physical address.

The tag field of the address is compared with the tags of all cache blocks simultaneously.

If a matching tag is found and the valid bit is 1, a cache hit occurs.

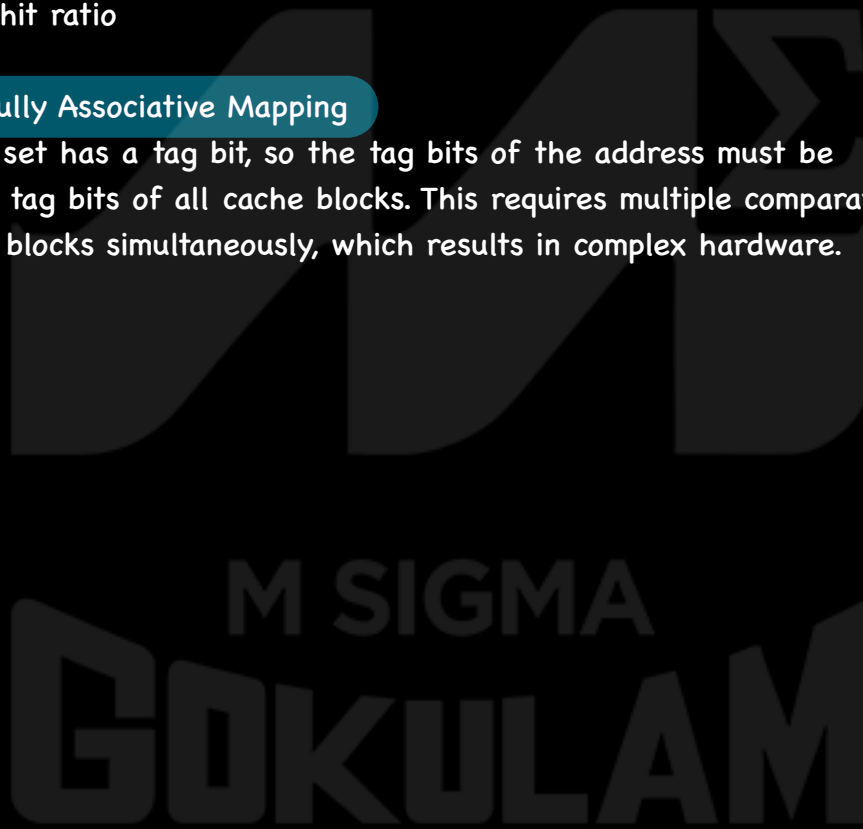
The required data is read from the corresponding cache block.

Advantages of Fully Associative Mapping

- Eliminates conflict misses
- Highest cache hit ratio

Disadvantages of Fully Associative Mapping

Each block in a set has a tag bit, so the tag bits of the address must be compared with the tag bits of all cache blocks. This requires multiple comparators to compare all the blocks simultaneously, which results in complex hardware.



Comparison of Cache Mapping Techniques			
Feature	Direct Mapping	Set Associative Mapping	Fully Associative Mapping
Block placement	Each main memory block maps to one specific cache block	Each main memory block maps to one set and can be placed in any block within that set	A main memory block can be placed in any cache block
Cache organisation	Cache divided into Sets (1 block per set)	Cache divided into sets, each set has 2 or more blocks	Cache consists one set with many block
Address fields	Tag, set bit, block Offset	Tag, set bit, block Offset	Tag, Block Offset
Conflict miss	High	Low	None
Cache hit ratio	Low	Medium	High
Hardware complexity	Very simple	Moderate	Very complex
Cost	Low	Medium	High
Access time	Fast	Moderate	Slower

Summary

- Direct mapping is simple and fast but suffers from high conflict misses.
- Set associative mapping reduces conflict misses and offers a balance between cost and performance.
- Fully associative mapping eliminates conflict misses completely but requires complex and expensive hardware.

Comparison of Cache Mapping Techniques			
Feature	Direct Mapping	Set Associative Mapping	Fully Associative Mapping
Block placement			
Cache organisation			
Address fields			
Conflict miss			
Cache hit ratio			
Hardware complexity			
Cost			
Access time			

- Explain the memory system architecture and memory hierarchy in detail.
- Explain cache memory and its basic concepts with a neat diagram.
- Describe direct mapping cache with address format and example.
- Explain set associative cache mapping using a 2-way set associative cache with a numerical example.
- Explain fully associative mapping with advantages and disadvantages.
- Compare all cache mapping techniques with suitable diagrams.
- Explain cache performance analysis and average access time.
- Discuss the problems of direct mapping and how they are solved using set associative mapping.
- A system has a main memory and cache. Given suitable parameters, calculate hit ratio and access time. (Numerical)

A digital computer has a main memory capacity of 128 MB and a cache memory of 1 MB. The cache uses 4-way set-associative mapping with a block size of 1024 bytes. Calculate the number of bits in the Tag, Set, and Word fields of the Main Memory address.

To which Set Number will the main memory address 0x3C4F5A be mapped?

$$MM = 128MB = 128 \times 2^{20} = 2^7 \times 2^{20} = 2^{27}$$

no: of bits to represent MM = $\log_2 2^{27} = 27$ bits.

$$CM = 1MB = 1 \times 2^{20} = 2^{20}$$

4 way set associative = no: of block in a set = 4

$$\text{Block size} = 1024B = 2^{10}$$

Need to find Main Memory Add.



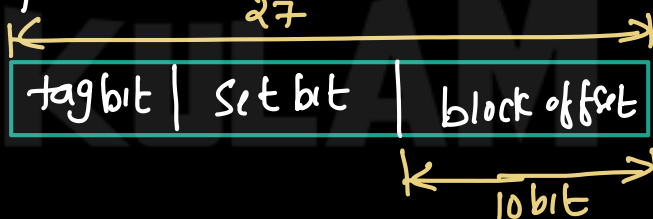
$$MM \text{ size} = 2^{27}$$

$$\text{no: of bit in MM Add} = \log_2 2^{27} = 27$$

Block offset: ?

$$\text{Block size} = 1024$$

$$\text{no: of bits to Represent Block} = \log_2 1024 = 10 \text{ bit}$$

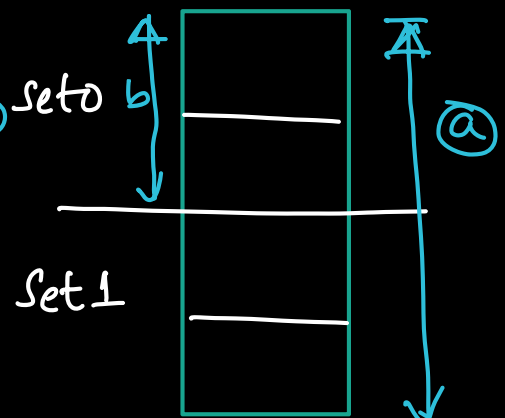


—Set bit ?

To find set bit, we need to find the no: of set in the cache

$$\text{no: of Set} = \frac{\text{Total No: of Block (a)}}{\text{no: of Block in a set (b)}}$$

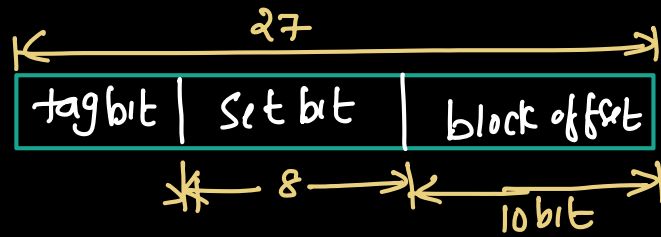
$$\begin{aligned} \text{Total No: of Blocks in cache} &= \frac{\text{Cache size}}{\text{block size}} \\ &= \frac{2^{20}}{2^{10}} = 2^{10} \end{aligned}$$



no: of set = $\frac{2^{10}}{4} = \underline{\underline{2^8}}$

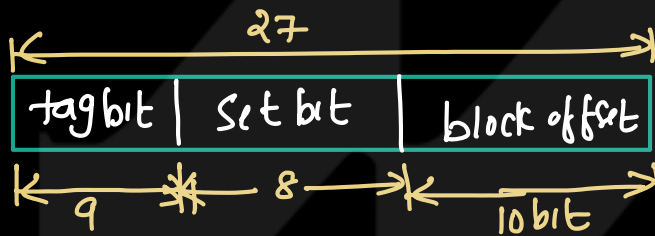
no: of bits to represent set = $\log_2 \text{no: of set}$
 $= \log_2 2^8 = \underline{\underline{8 \text{ bits}}}$

(N)



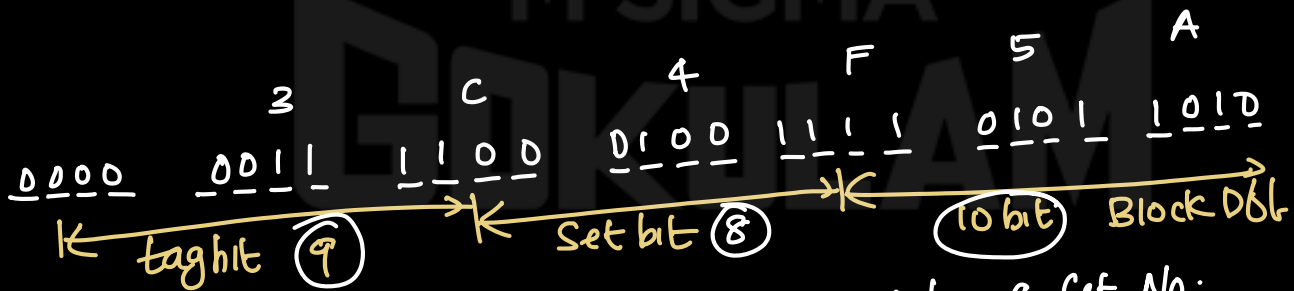
— tag bit = ?

tag bit = Total bits — set bit — block offset
 $= 27 - 8 - 10 = \underline{\underline{9 \text{ bit}}}$



(b) Memory Add is 27 bit.

Given Add is 0X 3C 4F 5A



Given Main Memory blocks mapped to a set No:

set bit $\rightarrow 00 \ 0100 \ 11 \rightarrow \underline{\underline{19}}$

Short-cut Set No = $\left(\frac{\text{Add}}{\text{Block size}} \right) \text{ mod no: of set}$

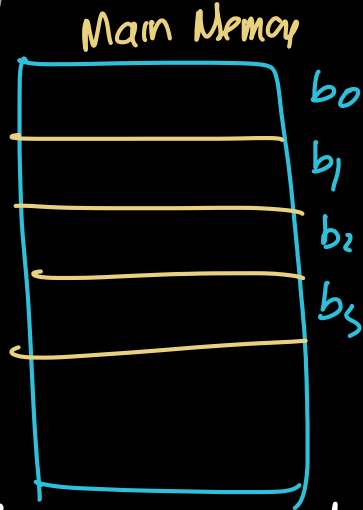
=

Cache Replacement policy

→ each memory block ^{in main memory} is mapped a set in cache.

Cache Memory

s ₀	b ₀
s ₁	b ₁
s ₂	b ₂
s ₂	b ₃

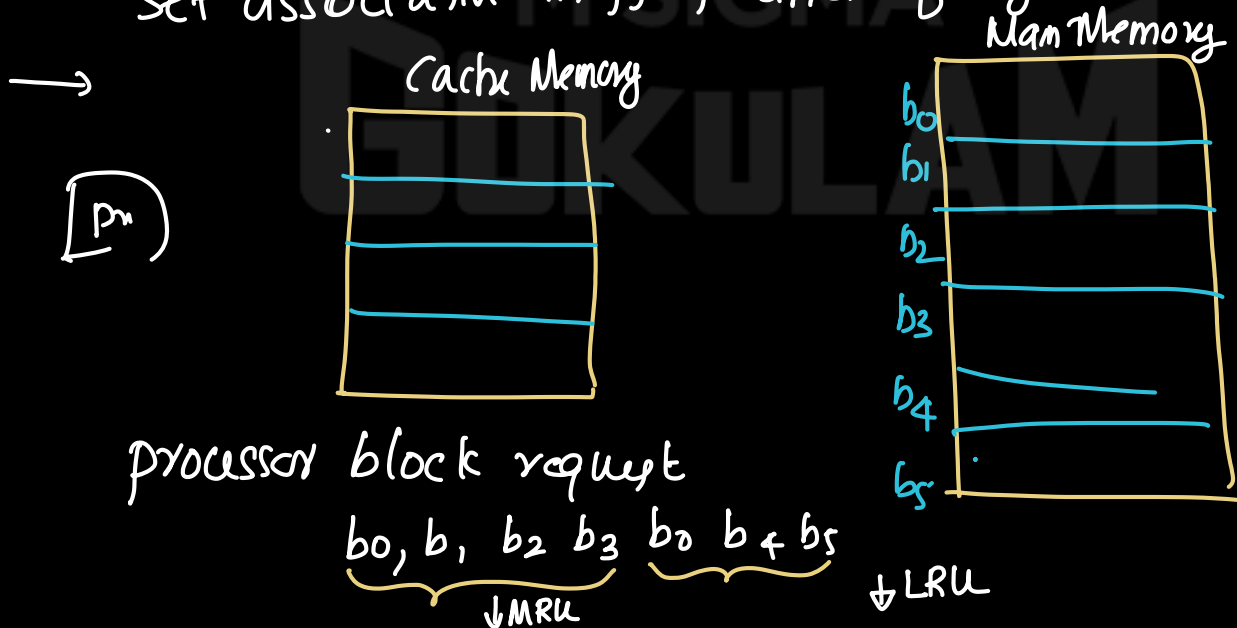


→ If cache completely filled and need to map new block, then need to replace a cache block. This is called replacement

→ Direct cache mapping → cache no cache replacement policy
 Set associative }
 full associative } Least Recently used
 LRU

LRU (Least Recently used)

→ it is a cache replacement policy used in set associative mapping and fully associating mapping



Block request

b₀

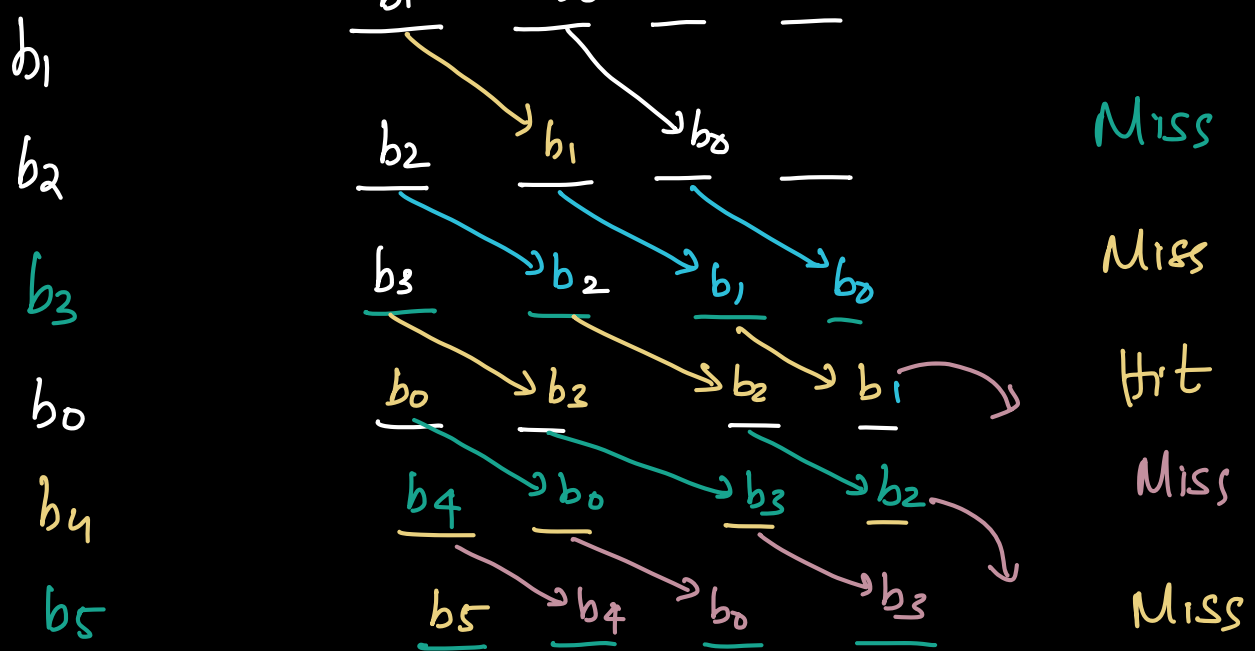
MRU LRU

b₀ — — —

b₁ → b₀

Miss

Miss



Cache Content $\rightarrow$ b5, b4, b0, b3

$$\text{Hit Rate} = \frac{\text{no. of hit}}{\text{Total no. of memory}} = \frac{1}{7} = 0.142 \times 100 = \underline{\underline{14.2\%}}$$

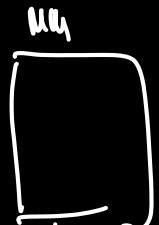
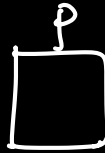
$$\text{Miss Rate} = \frac{\text{no. of Miss}}{\text{Total no. of m}} = \frac{6}{7} = 0.857 = \underline{\underline{85.7\%}}$$

M SIGMA
GOKULAM

different types of cache Miss

① Compulsory Miss →

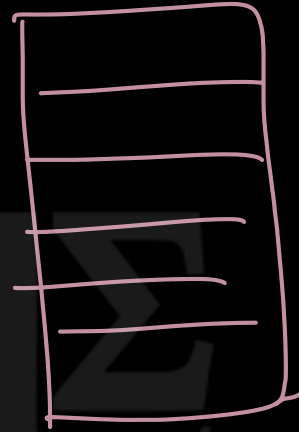
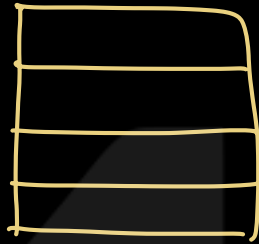
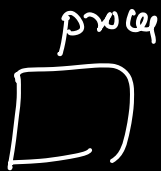
when processor access cache for the first time



② Conflict Miss

③ Capacity Miss → due to small size in

① Compulsory Miss

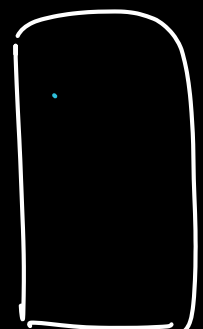
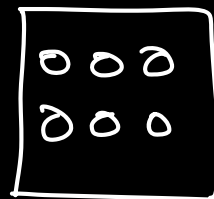


processor request → b0

How we can reduce Cache Misses

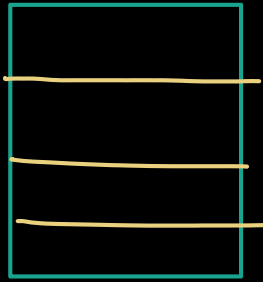
	Compulsory	Conflict	Capacity
Cache size ↑	—	↓	↓
Block size ↑	↓	↑	—
associative ↑ (N)	—	↓	—

Compulsory ↓



(

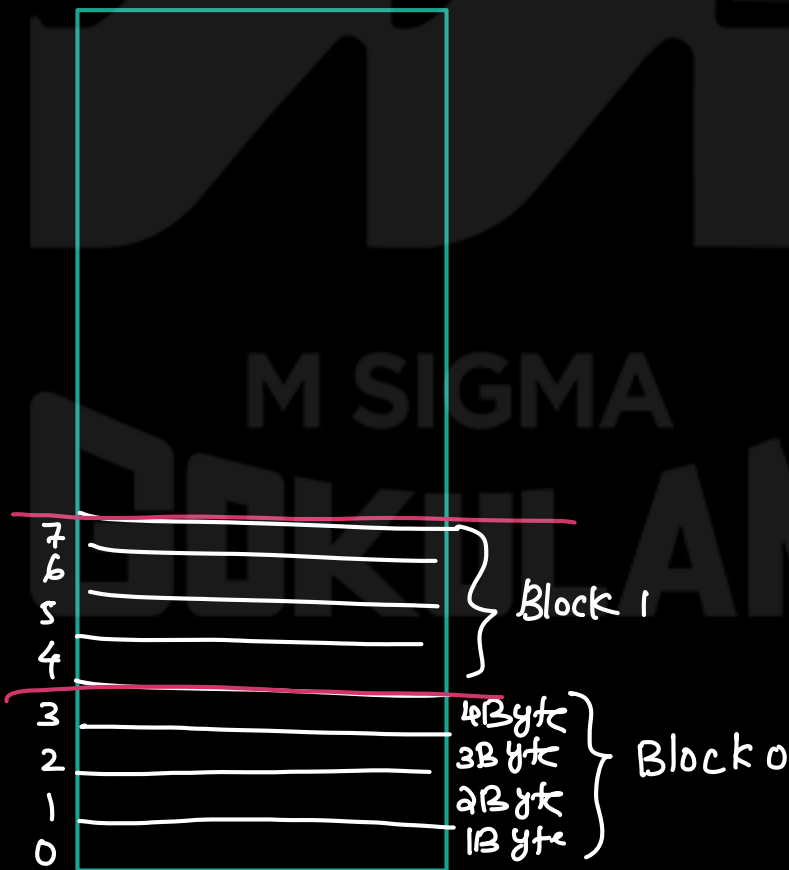
A computer system has a cache memory with a total capacity of 4 blocks. The block size is 4 bytes. The cache is initially empty and uses the LRU (Least Recently Used) replacement policy. The CPU generates the following sequence of Byte Addresses : 4, 8, 1, 12, 4, 16, 20, 8, 4, 0, 10, 12. Calculate the final Hit Ratio. Show the status of cache line for each byte address access.



no. of block = 4
Block size = 4 Byte

cpu generate Add 4, 8, 1, 12, 4, ...

Main Memory divided into Block, Block size of both cache and main memory are same 4-Byte.



Addr

Memory Add btw 0 to 3 is block 0.
4 - 7 is block 1

main memory Address

0 - 3 → block 0
4 - 7 → block 1
8 - 11 → block 2
12 - 15 → block 3

16 — 19 —————> block 4
20 — 23 —————> block 5

the have given the memory Address, from the Address we need to find the block no of main Memory

eg: Main Memory Add 4 —————> block 1

(i) Convert byte Address to block no:

byte Addr	Block NO
4	1
8	2
1	0
12	3
4	1
16	4
20	5
8	2
4	1
0	0
10	2
12	3

Logic —> If hit occurs bring that block to MRU
If cache is full, and Miss occurs replace LRU blocks

Block No

cache state

Hit/Miss

	MRU		LRU	
	↓		↓	
1	1	—	—	Miss
2	2	1	—	Miss
0	0	2	1	Miss
3	3	0	2	Miss
1	1	3	0	Hit
4	4	1	3	Miss
5	5	4	1	Miss
2	2	5	4	Miss
1	1	2	5	Hit
0	0	1	2	Miss
2	2	0	1	Hit
3	3	2	0	Miss

Bring the 1
to MRUReplace LRU
(2)Replace LRU
(0)

Bring 1 to MRU

Replace LRU (5)

Bring 2 to MRU

Replace LRU (5)

Total access = 12

no. of Hit = 3

no. of Miss = 9

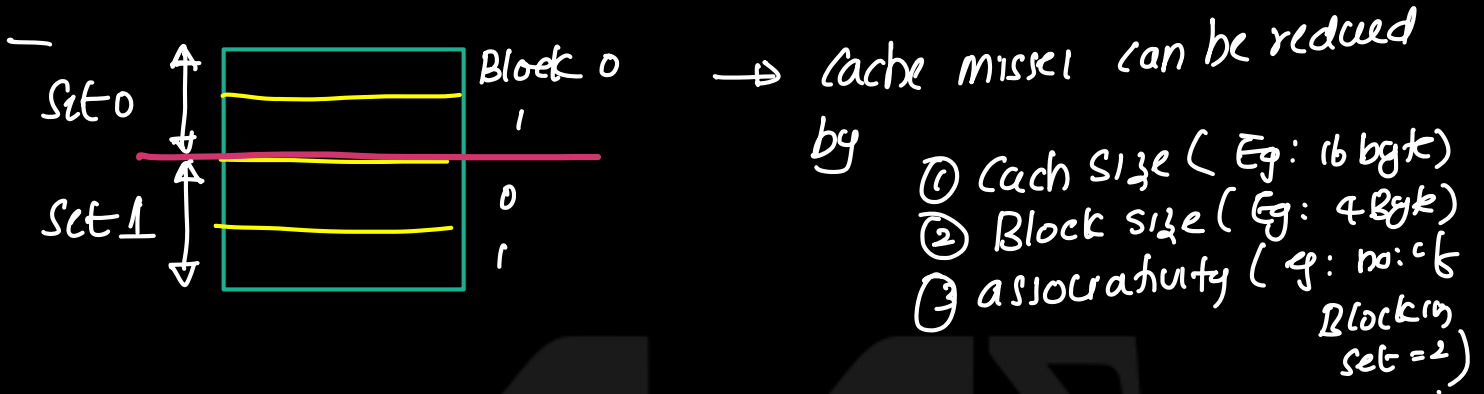
$$HR = \frac{3}{12} = \frac{1}{4} = \underline{\underline{25\%}}$$

Reducing cache Misses

Note

— Cache performance is affected by Miss Rate

$$\text{Miss Rate} = MR = \frac{\text{no: of Misses}}{\text{Total No: of access.}}$$

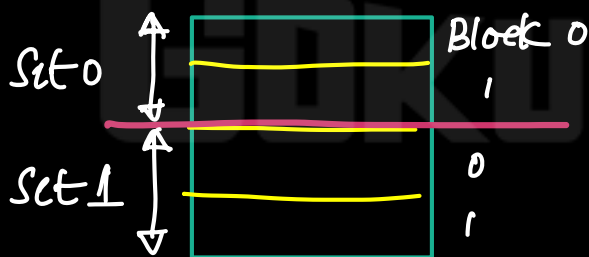


— types of Misses

- ① Compulsory Misses
- ② Capacity Misses
- ③ Conflict Misses

① Compulsory Misses

→ Let say the block request is b_0, b_1



initially block has not mapped.

— initially no block are not mapped to cache.
→ so when block request b_0 it is not present in the cache. thus a miss occur. this miss is called compulsory miss

— this miss occur when memory accessed for the first time.

— increasing block size reduces the compulsory Misses

② Conflict Miss.

- This occurs when multiple memory blocks map to the same cache set, evict block that are still needed

eg: Block request b_0, b_4, b_0, b_4



→ First request b_0 — cache hit
→ 2nd Request b_4 → Replace b_0 (Miss)

→ $\cancel{b_0} \ b_4$
1 2

3rd Request b_0 → Miss (already b_0 present but replaced)

$\cancel{b_0}, \cancel{b_4}, b_0$

- Conflict Miss can be reduced by increasing the cache capacity.
(cache capacity ↑, no. of hit ↑)

③ Capacity Misses

- This miss occurs when the cache size is too small to hold all the data blocks that are needed
- Compulsory Miss can be reduced by increasing cache size

Effect of cache parameter on Cache Miss (Note)

Cache parameter .

- ① Cache capacity
- ② Block size
- ③ Associativity

- ① **Cache capacity**
- increase in cache capacity reduces capacity miss and reduce conflict miss
 - Does not affect compulsory Misses.

- ② **Block size**
- increasing block size reduces the compulsory misses.
 - increase block size increases conflict misses

- ③ **Associativity**
- increasing associativity means, increasing the no. of blocks in a set
 - For small cache increasing the associativity, decreases the conflict misses

Summary

	Compulsory	Conflict	Capacity
Cache size ↑	—	↓ ②	↓ ①
block size ↑	↓ ②	↑	—
associativity ↑	—	↓	—

Module -3

N

A processor's cache is redesigned from Direct-Mapped to 2-Way Set Associative while keeping the total cache capacity and block size constant. Identify which type of cache miss is primarily reduced by this change. Justify your answer by explaining why the other two types of misses remain unaffected.

The type of miss primarily reduced is: Conflict Miss

Reason: Conflict misses occur when multiple memory blocks compete for the same cache line (set). In a direct-mapped cache, each memory block maps to exactly one cache line, so two frequently used blocks mapping to the same line will keep evicting each other. Increasing associativity to 2-way gives each set two slots, so two competing blocks can coexist – directly reducing conflict misses.

Why the other two miss types remain unaffected:

Compulsory (Cold) Misses: These occur when a block is accessed for the first time and is never in the cache regardless of associativity or capacity.

Capacity Misses: These occur because the cache is too small to hold all working-set data. Since total cache size is unchanged, capacity misses are unaffected.

M SIGMA
GOKULAM

Sample questions

Explain the different types of cache misses and methods to reduce them.

Discuss how cache capacity, block size, and associativity affect cache miss rate.

Explain cache miss behavior

